



Predictive Maintenance Analytics for Industrial Equipment

by

Bruno Mascarenhas da Silva

Chandan Rana

Faizanfarid Malek

Roshan Tuladhar

Field Project

Submitted in partial fulfillment of the requirements for the
Master of Data Analytics

Graduate Studies
University of Niagara Falls

Niagara Fall, ON

March 2026

Signature Page

Date: _____

This field project paper has been examined and approved.

Review Committee:

Professor Hany Osman, Faculty Advisor

Approved:

Associate Dean

Abstract

Unexpected equipment failures remain one of the most disruptive and costly challenges in modern manufacturing. Unplanned machine breakdowns can halt production lines, increase maintenance costs, and create potential safety risks. Traditional maintenance strategies, such as reactive repairs or fixed maintenance schedules, often fail to reflect the true degradation patterns of industrial machinery. This project investigates how advanced data analytics techniques can support predictive maintenance by identifying machines at risk of failure before breakdown occurs.

The analysis uses the AI4I 2020 Predictive Maintenance Dataset from Kaggle, which contains 10,000 observations of machine operating conditions measured across five sensor variables: air temperature, process temperature, rotational speed, torque, and tool wear. The dataset also includes machine type categories and a binary machine failure indicator with five specific failure modes. Because machine failures represent only 3.39% of the observations, the prediction task presents a significant class imbalance challenge typical of real-world industrial systems.

The analytical pipeline included exploratory data analysis, data cleaning, and statistical hypothesis testing. Welch's t-tests and Cohen's d effect size measurements were applied to identify significant differences between failure and non-failure machines. Several classification models were trained and evaluated, including Logistic Regression, Random Forest, Gradient Boosting, XGBoost, and a Multilayer Perceptron neural network. Ensemble models using Soft Voting and Stacking were also implemented to improve predictive performance under class imbalance conditions. The Weighted Soft Voting with calibrated RF ensemble with isotonic

probability calibration was selected for deployment, achieving a PR-AUC of 0.87 and a tuned threshold providing over 95% recall.

In addition to classification modeling, SHAP-based explainability analysis was applied to interpret model predictions and identify key drivers of machine failure risk. A regression-based Remaining Useful Life (RUL) model was developed to estimate time to failure, and survival analysis using Kaplan–Meier curves and Cox proportional hazards models was conducted to analyze failure probability over time. These analytical components were integrated into a three-page interactive Streamlit dashboard that enables real-time risk monitoring, scenario testing, and survival curve visualization.

The results demonstrate that a data-driven predictive maintenance framework can reliably detect high-risk machines, provide interpretable insights into failure mechanisms, and support proactive maintenance planning. The approach aligns with Industry 4.0 principles and offers a practical decision-support tool for maintenance planners and operations managers seeking transition from reactive to predictive maintenance strategies.

Acknowledgments

We would like to express our sincere gratitude to Dr. Hany Osman for his guidance, feedback, and encouragement throughout this capstone project. His expertise in data analytics and his willingness to challenge our thinking pushed us to deliver work we are genuinely proud of.

We also thank the University of Niagara Falls Canada and the Master of Data Analytics program for providing the academic environment, tools, and resources that made this research possible.

A special thank you to the open-source community and the contributors behind scikit learn, Streamlit, SHAP, lifelines, and the many Python libraries that powered this project. We are grateful to the creators of the AI4I 2020 Predictive Maintenance Dataset on Kaggle for making real world industrial data accessible for academic research.

Finally, we thank our families and friends for their patience and support during the long hours that went into building, debugging, and refining this system.

Table of Contents

Abstract	3
List of Tables	9
List of Figueres	10
Chapter I: Introduction.....	11
Problem Identification	11
Research Question and Hypotheses	12
Chapter II: Data Collection and Preparation.....	14
Data Collection	14
Data Cleaning.....	15
Feature Engineering.....	16
Chapter III: Statistical Analysis and Diagnostics	18
Hypothesis Testing.....	18
Hypothesis 1: Temperature and Load Interaction.....	18
Hypothesis 2: Sensor Association.....	20
Hypothesis 3: Model Performance.....	21
Diagnostics and Validation	22
Chapter IV: Analytics Applications.....	24
Baseline Classification Models.....	24
Ensemble Methods and Model Selection.....	25

SMOTE Imbalance Experiment.....	27
SHAP Model Explainability	27
RUL/TTF Regression Estimation	29
Survival Analysis	30
Chapter V: Data Visualization	33
Streamlit Dashboard Overview	33
Page 1: Simulation Dashboard.....	33
Automatic Maintenance at 90% Risk	34
High Risk Alerts and Maintenance Tables	35
CSV Snapshot Save and Resume.....	36
Machine Selection.....	37
Page 2: What If Analysis	37
Page 3: Survival Analysis	38
Chapter VI: Findings and Discussion	39
Findings.....	39
Discussion	39
Chapter VII: Conclusion and Recommendations	41
Conclusion	41
Recommendations.....	41
Chapter VIII: Project Management.....	43

Agile Methodology and Taiga.io	43
Sprint Overview	43
Version Control with GitHub.....	44
References	45
Appendix A: Project Repository Structure	48

List of Tables

Table 1: Dataset Feature Descriptions	14
Table 2: Logistic Regression with Interaction Term (Hypothesis 1).....	19
Table 3: Levene's Test for Equal Variances	20
Table 4: One Way ANOVA and Welch ANOVA Results (Hypothesis 2).....	20
Table 5: Friedman Test Results (Hypothesis 3).....	21
Table 6: Mean PR AUC by Model Group	22
Table 7: Baseline Model Comparison.....	25
Table 8: Full Ensemble Comparison.....	26
Table 9: Maintenance Duration by Machine Type	35
Table 10: Agile Sprint Timeline	43

List of Figueres

Figure 1: Dataset preview showing the first five observations.....	15
Figure 2: Interaction effect plot showing diverging curves for low vs high torque	19
Figure 3: Ensemble model comparison showing PR AUC and Recall side by side.....	Error!
Bookmark not defined.	
Figure 4: SHAP summary plot of key features affecting failure probability.....	28
Figure 5: SHAP Summary Plot showing global feature importance and directionality	29
Figure 6: SHAP Summary Plot showing global feature importance and directionality	30
Figure 7: Kaplan Meier Survival Curves Stratified by Machine Type	31
Figure 8: Survival Model Comparison	32
Figure 9: Fleet monitoring tables showing high risk machines and maintenance status	36
Figure 10: What If Analysis with editable sensor values	38
Figure 11: Survival Analysis with KM and Cox panels	38
Figure 12: Taiga Sprint Taskboard Overview	44

Chapter I: Introduction

Problem Identification

In many industrial environments, maintenance strategies are still largely limited to either reactive (run-to-failure) or preventive (schedule-based) approaches. Reactive maintenance, while simple to implement, incurs significant costs due to unplanned downtime, emergency repairs, and cascading operational disruptions across production systems (Cotton Gins Blog, 2022). Preventive maintenance, although more structured, often results in unnecessary servicing of equipment that is still functioning properly, leading to inefficient use of labor and resources (IBM, 2023). As a result, both approaches are inherently suboptimal in terms of cost and operational efficiency. Prior research has identified these limitations, highlighting that traditional maintenance strategies suffer from high costs and inefficiencies, thereby motivating the transition toward predictive, data-driven maintenance paradigms (Zhu et al., 2019).

A fundamental challenge in predictive maintenance is the lack of reliable, data-driven methods capable of identifying early warning signals of machine failure with sufficient lead time for intervention. This problem is further exacerbated by the inherent class imbalance present in industrial datasets, where failure events typically represent less than 5% of all observations. Under such conditions, a naive classifier that predicts the majority class (i.e., no failure) can achieve high overall accuracy while failing to detect any true failure cases. Consequently, traditional accuracy metrics become misleading, and models trained without explicitly addressing class imbalance tend to be biased toward the majority class, neglecting the rare but critical failure events they are intended to identify (He & Garcia, 2009; King & Zeng, 2001).

Compounding the problem, many predictive systems that do exist operate as black boxes. They produce a risk score or a binary flag without any explanation of why a particular machine was flagged. For a maintenance engineer standing on a factory floor, a system that says "this machine will fail" without saying "because torque has been climbing while RPM is dropping" is not particularly useful. Without interpretability, trust erodes and adoption stalls.

This project addresses all three of these gaps: the need for accurate failure prediction under class imbalance, the need for transparent and explainable predictions, and the need for an accessible, interactive system that maintenance personnel can use in practice.

Research Question and Hypotheses

“Which machine learning approach provides the best precision recall tradeoff for predicting machine failure under class imbalance, and which sensor variables most strongly drive failure risk in the AI4I 2020 dataset?”

To operationalize this question, three hypotheses were formulated:

Hypotheses 1: Temperature and Load Interaction

H0: The temperature difference between process temperature and air temperature and torque has no significant effect on machine failure probability.

H1: The temperature difference between process temperature and air temperature and torque significantly affects machine failure probability.

Hypotheses 2: Sensor Association

H0: Sensor variables such as torque, rotational speed, temperature, and tool wear have no significant association with machine failure.

H1: Sensor variables such as torque, rotational speed, temperature, and tool wear are significantly associated with machine failure.

Hypotheses 3: Model Performance

H0: Ensemble machine learning models do not significantly outperform logistic regression and deep learning models in predicting machine failure.

H1: Ensemble machine learning models outperform logistic regression and deep learning models in predicting machine failure.

Chapter II: Data Collection and Preparation

Data Collection

The dataset used in this study is the AI4I 2020 Predictive Maintenance Dataset, sourced from Kaggle. It was designed to reflect realistic industrial operating conditions and contains 10,000 observations of machine sensor readings. Each observation represents a snapshot of a machine's operating state at a given point in time, captured across 14 variables.

The dataset includes five core sensor measurements: air temperature (measured in Kelvin), process temperature (Kelvin), rotational speed (RPM), torque (Newton meters), and tool wear (minutes of accumulated use). A categorical variable identifies the product quality type as High (H), Medium (M), or Low (L). The binary target variable, Machine failure, indicates whether a failure event occurred (1) or not (0). Five additional binary columns (TWF for Tool Wear Failure, HDF for Heat Dissipation Failure, PWF for Power Failure, OSF for Overstrain Failure, and RNF for Random Failure) identify the specific failure mode when a failure did occur.

Table 1: Dataset Feature Descriptions

Feature	Type	Description
UDI	Integer	Unique row identifier (1 to 10000)
Product ID	String	Machine identifier (e.g., M14860)
Type	Categorical	Product quality tier: H, M, or L
Air temperature [K]	Float	Ambient air temperature in Kelvin
Process temperature [K]	Float	Internal process temperature in Kelvin
Rotational speed [rpm]	Integer	Machine rotational speed
Torque [Nm]	Float	Mechanical torque applied
Tool wear [min]	Integer	Cumulative tool usage time
Machine failure	Binary	Target: 1 = failure, 0 = no failure
TWF, HDF, PWF, OSF, RNF	Binary	Failure mode flags (excluded as leakage)

The dataset is well structured and publicly available, making it suitable for supervised machine learning research. The failure rate of 3.39% (339 failures out of 10,000 observations) mirrors real world industrial conditions where breakdowns are relatively uncommon but operationally significant.

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	0	0	0	0	0

Figure 1: Dataset preview showing the first five observations

The AI4I 2020 dataset is well suited for predictive maintenance research because it contains both operational sensor measurements and labeled failure events. These characteristics allow the application of machine learning techniques to detect patterns associated with machine breakdowns. Additionally, the dataset includes multiple sensor variables that represent realistic industrial monitoring conditions, making it appropriate for evaluating predictive maintenance models.

Data Cleaning

Data cleaning was conducted systematically to validate the dataset's integrity before any modeling work began. The idea was straightforward: check everything, fix only what is actually broken, and preserve the original data.

Data preprocessing included the following steps:

1. Removal of non-informative identifiers
2. Verification of missing values and duplicates
3. Creation of engineered variables (e.g., temperature difference)

4. Standardization of numerical variables for model compatibility
5. Stratified train–test split to preserve class distribution

The cleaning process covered four key areas. First, the target variable was inspected to confirm it contained only valid binary values (0 and 1) with no unexpected entries. The distribution showed 9,661 non-failure cases and 339 failure events, confirming a heavily imbalanced classification problem. Because machine failures represent only a small proportion of the dataset, the classification task presents a severe class imbalance problem. In such situations, traditional accuracy metrics can be misleading because a model that predicts “no failure” for all observations may still achieve high accuracy. Therefore, evaluation of metrics such as precision, recall, and PR-AUC were prioritized.

Second, a comprehensive missing value check was performed across all 14 columns, and **no missing** values were detected. Third, duplicate records were checked and **zero duplicates** were found. Fourth, sensor variables were validated for physically implausible values such as non-positive temperatures or negative torque, and all checks returned **zero violations**.

Based on these results, the cleaned dataset was saved as **ai4i2020_cleaned.csv** with all 10,000 original observations preserved. No rows were removed, and no values were altered. Extreme values observed during EDA were retained because they represent genuine operating conditions rather than data errors.

Feature Engineering

Two physically meaningful features were engineered to capture domain specific failure mechanisms that the raw sensor readings alone might not express clearly:

Temp_diff (Process temperature minus Air temperature): A new feature, temperature difference, was created by subtracting air temperature from process temperature. This engineered variable captures abnormal thermal conditions that may signal mechanical stress or cooling inefficiencies in the system.

This captures the thermal gradient across the machine. When this difference narrows or behaves abnormally under load, it may indicate heat dissipation problems directly related to the HDF failure mode documented in the dataset.

Torque_RPM_ratio (Torque divided by Rotational speed + 1): This ratio captures mechanical strain per unit of rotational speed. A machine running high torque at low RPM is under significant stress, which is a known precursor to overstrain failure. The +1 in the denominator prevents division by zero.

These features were implemented in the centralized features.py module and applied consistently across all notebooks and the deployment pipeline through the add_engineered_features() function.

Chapter III: Statistical Analysis and Diagnostics

Hypothesis Testing

The tests were selected to match the specific structure of each hypothesis: logistic regression with an interaction term for the conditional Hypothesis 1, One Way ANOVA for the group comparison in Hypothesis 2, and the Friedman test for the multi model ranking in Hypothesis 3.

Hypothesis 1: Temperature and Load Interaction

Hypothesis 1 examines whether the combination of a narrow thermal gradient and high mechanical load significantly predicts failure. Because this hypothesis is specifically about a combined effect (low Temp_diff AND high Torque together), a plain t test or ANOVA would not capture the joint relationship. Logistic regression with an interaction term was used instead, as it directly models whether the product of Temp_diff and Torque significantly predicts failure probability (Bruce et al., 2020).

Both variables were standardized before fitting to ensure the interaction term is on a comparable scale, and the coefficients are interpretable. The model specification was $\text{failure} \sim \text{Temp_diff} + \text{Torque} + \text{Temp_diff} \times \text{Torque}$.

Table 2: Logistic Regression with Interaction Term (Hypothesis 1)

Variable	Coefficient	Odds Ratio	p value
Intercept	-4.0426	0.0176	< 0.001
Temp_diff	-0.5283	0.5896	< 0.001
Torque	1.0595	2.8848	< 0.001
Interaction (Temp_diff x Torque)	-0.1702	0.8435	0.0116

The interaction term is statistically significant with a p-value of 0.0116 (< 0.05). The negative coefficient (-0.1702) and odds ratio below 1.0 (0.8435) indicate that as Temp_diff decreases while Torque increases, the probability of failure rises. This confirms that the combined effect of narrowing thermal gradient under high mechanical load increases failure risk, consistent with the heat dissipation failure mechanism described in the dataset documentation.

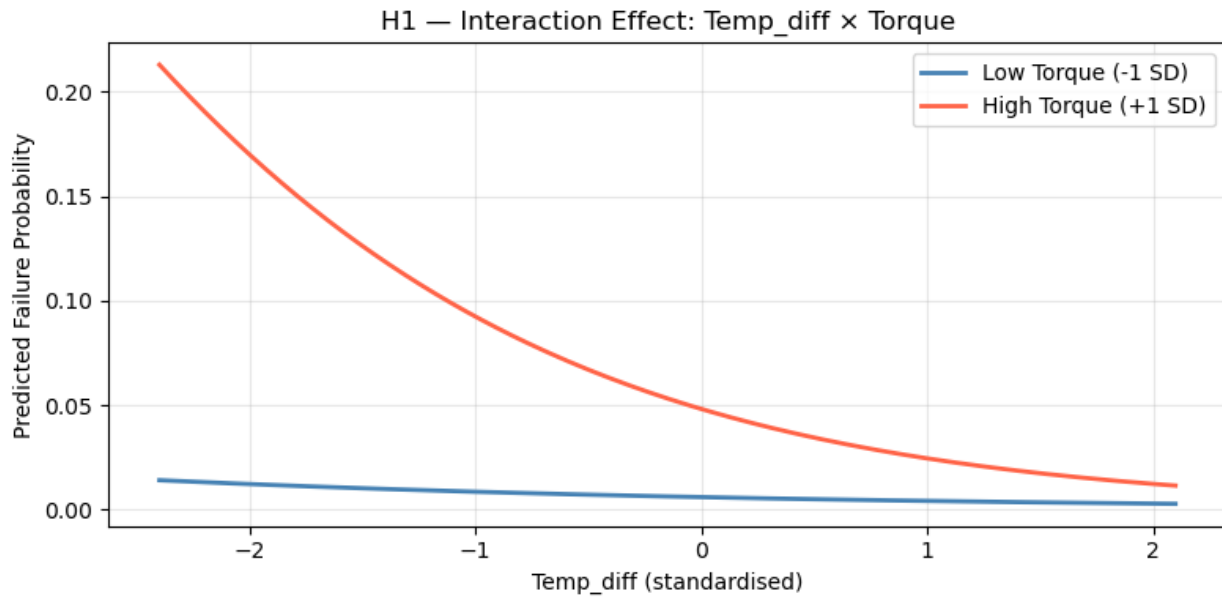


Figure 2: Interaction effect plot showing diverging curves for low vs high torque

Decision: Reject H0. H1 is confirmed. The interaction between temperature difference and torque significantly predicts failure probability (p-value = 0.0116).

Hypothesis 2: Sensor Association

Hypothesis 2 tests whether the five sensor variables are significantly associated with machine failure. One Way ANOVA was conducted for each sensor variable, comparing the mean values between failed ($n = 339$) and non-failed ($n = 9,661$) machines. Levene's test for equal variances was run first to determine whether standard ANOVA or Welch ANOVA was appropriate.

Table 3: Levene's Test for Equal Variances

Variable	p value	Variance Equality
Air temperature [K]	0.3349	Equal
Process temperature [K]	0.0001	Not equal (Welch used)
Rotational speed [rpm]	< 0.001	Not equal (Welch used)
Torque [Nm]	< 0.001	Not equal (Welch used)
Tool wear [min]	< 0.001	Not equal (Welch used)

Four of the five variables showed unequal variances, indicating that Welch ANOVA (which does not assume equal variances) was the appropriate test for those variables.

Table 4: One Way ANOVA and Welch ANOVA Results (Hypothesis 2)

Variable	Test Used	F-Statistics	p value	Eta ²
Air temperature [K]	One Way ANOVA	68.6084	< 0.001	0.0068
Process temperature [K]	Welch ANOVA	15.1974	< 0.001	0.0013
Rotational speed [rpm]	Welch ANOVA	4.3546	0.0376	0.0020
Torque [Nm]	Welch ANOVA	138.7875	< 0.001	0.0366
Tool wear [min]	Welch ANOVA	85.8271	< 0.001	0.0111

All five sensor variables showed statistically significant differences between failure and non-failure groups (all p values well below 0.05). Torque showed the largest F statistic (138.78) and the largest effect size (Eta squared = 0.0366), indicating that failed machines operate under substantially higher mechanical load. Tool wear showed the second largest effect (Eta squared = 0.0111), consistent with the intuition that machines with more accumulated wear are closer to failure.

The Eta squared values are small in absolute terms, which is expected given the highly imbalanced dataset where failures represent only 3.39% of observations. In such datasets, even variables with strong practical significance can produce small Eta squared values because the overwhelming majority of variance comes from the large non failure group.

Decision: Reject H0. H1 is confirmed. Sensor variables such as torque, rotational speed, temperature, and tool wear are significantly associated with machine failure.

Hypothesis 3: Model Performance

Hypothesis 3 tests whether ensemble models outperform logistic regression and deep learning models on precision recall metrics. The Friedman test was used as a non-parametric equivalent of repeated measures ANOVA to test whether at least one model ranks significantly differently from the rest across all evaluation metrics. The test was applied across 12 models (Logistic Regression, Random Forest, Gradient Boosting, XGBoost, MLP, TabNet, LSTM, Soft Voting equal and weighted, Stacking LR and GB meta, and the calibrated deployment model) and 5 metrics (PR AUC, ROC AUC, Recall, Precision, F1).

Table 5: Friedman Test Results (Hypothesis 3)

Metric	Value
Chi squared statistic	28.2000
Degrees of freedom	4
p value	0.000011
Significance	$p < 0.001$

Table 6: Mean PR AUC by Model Group

Model Group	Mean PR AUC	vs LR
Ensemble models	0.8700	+94.9%
Deep Learning (MLP, TabNet, LSTM)	0.7056	+58.1%
Logistic Regression	0.4463	baseline

The Friedman test confirms that model performances are not equivalent ($p < 0.001$). Ensemble models achieve a mean PR AUC of 0.8700, representing a 94.9% improvement over Logistic Regression (0.4463) and outperforming deep learning models (mean PR AUC 0.7056). This confirms that ensemble methods, specifically Weighted Soft Voting with calibrated RF, deliver the strongest overall performance for failure prediction under class imbalance.

Decision: Reject H_0 . H_1 is confirmed. Ensemble models achieve superior precision recall performance compared to both logistic regression and deep learning alternatives.

Diagnostics and Validation

Several diagnostic procedures were applied throughout the modeling pipeline to ensure robustness and generalizability. Stratified train test splitting (80/20) was employed to preserve the observed failure rate (3.39%) in both training and testing sets, ensuring representative class distributions (Hastie et al., 2009). Model validation was further conducted using 5 fold stratified cross validation, a widely adopted approach for assessing model stability across different data partitions. The Weighted Soft Voting with calibrated RF ensemble demonstrated consistent PR AUC scores across all folds, indicating that the observed performance is not attributable to a favorable single data split but rather reflects stable predictive capability.

For the survival analysis component, the Cox Proportional Hazards model was applied, and its underlying assumptions were evaluated using standard diagnostic procedures available in the lifelines library. Given the rarity of machine failure events in the dataset, the classification task is inherently imbalanced. In such settings, Precision-Recall AUC (PR AUC) is a more informative evaluation metric than ROC AUC or overall accuracy, as it emphasizes the model's ability to correctly identify rare positive instances (He and Garcia, 2009).

The statistical analysis and validation procedures described in this chapter establish the reliability of the predictive modeling framework. The following chapter expands the analysis by applying multiple analytical techniques, including ensemble learning, explainability analysis, regression based remaining useful life estimation, and survival analysis.

Chapter IV: Analytics Applications

This chapter presents the analytical methods applied to address the predictive maintenance problem. Multiple analytics techniques were implemented to evaluate machine failure risk from different perspectives. These methods include classification modeling for failure prediction, ensemble learning for improved performance, SHAP explainability for model interpretation, regression-based estimation of remaining useful life, and survival analysis to estimate the probability of machine failure over time.

This chapter documents the six analytical methods applied to address the research question: baseline classification, ensemble modeling, SMOTE imbalance experimentation, SHAP based explainability, RUL/TTF regression estimation, and survival analysis.

Baseline Classification Models

Six baseline models were evaluated in Notebook 06 to establish a performance floor: Logistic Regression (`class_weight = "balanced"`), Random Forest (300 trees, balanced subsampling), Gradient Boosting, and a Multilayer Perceptron (MLP). Evaluation focused on Precision Recall AUC rather than ROC AUC because PR AUC is more informative for imbalanced datasets.

These models were selected to represent distinct machine learning paradigms. Logistic Regression serves as a simple and interpretable linear baseline (Hastie et al., 2009), while ensemble methods such as Random Forest (Breiman, 2001) and Gradient Boosting (Friedman, 2001) are capable of capturing complex nonlinear relationships through combinations of decision trees. In addition, neural network models provide highly flexible function approximators that can model intricate nonlinear interactions among input features, such as sensor data (Goodfellow et al., 2016).

In addition, two more deep learning baselines, TabNet and LSTM, were evaluated to extend the comparison beyond traditional machine learning models. TabNet was included because it is specifically designed for tabular data and can learn both feature selection and representation jointly, while LSTM was explored as a more advanced neural network architecture to test whether sequential-style learning could capture deeper patterns in the sensor features. Including these models allowed the study to compare conventional machine learning approaches with more flexible deep learning methods for failure prediction.

Table 7: Baseline Model Comparison

Model	PR AUC	ROC AUC	Recall	F1
Logistic Regression	0.446	0.934	0.868	0.296
Random Forest	0.826	0.955	0.676	0.780
Gradient Boosting	0.870	0.972	0.765	0.832
MLP (Neural Network)	0.820	0.967	0.809	0.763
Tabnet	0.696	0.961	0.852	0.417
LSTM	0.599	0.963	0.897	0.389

Ensemble Methods and Model Selection

Ensemble learning combines predictions from multiple models to improve predictive accuracy and robustness. By aggregating the strengths of diverse algorithms, ensemble methods can reduce variance and bias, often resulting in superior performance compared to individual models (Hastie et al., 2009). Such approaches are particularly effective in complex classification tasks, where capturing nonlinear relationships and interactions is critical, as is often the case in predictive maintenance applications (Friedman, 2001). Notebook 07 extended the analysis to ensemble strategies. Five approaches were compared: Soft Voting with equal weights, Soft Voting with PR

AUC derived weights, Weighted Soft Voting with calibrated RF, Stacking with a Logistic Regression meta learner, and Stacking with a Gradient Boosting meta learner.

The Weighted Soft Voting with calibrated RF ensemble was selected for deployment. This decision was driven by practical deployment considerations related to probability calibration and interpretability. This model produced well-distributed probability estimates (ranging from near 0.0 to 0.96), enabling a meaningful continuous risk representation in the operational dashboard. Well-calibrated and properly distributed probability estimates are critical for decision-making systems, as poorly calibrated models may produce unreliable or uninformative confidence scores. Additionally, simpler ensemble strategies such as averaging (Soft Voting) often provide more stable and interpretable probability outputs compared to more complex meta-model approaches like stacking (Hastie et al., 2009; Géron, 2019).

The deployed model uses isotonic calibration on the Random Forest component, with base estimators RF, GB, and MLP and weights of [2, 2, 1] giving RF and GB higher influence. The threshold was tuned to achieve at least 95% recall.

Table 8: Full Ensemble Comparison

Model	PR AUC	ROC AUC	Recall	Prec.	F1
Stacking (LR meta)	0.872	0.974	0.853	0.569	0.682
SoftVoting (PR AUC wt)	0.870	0.975	0.824	0.718	0.868
SoftVoting (equal wt)	0.870	0.975	0.824	0.718	0.868
Stacking (GB meta)	0.869	0.965	0.809	0.750	0.859

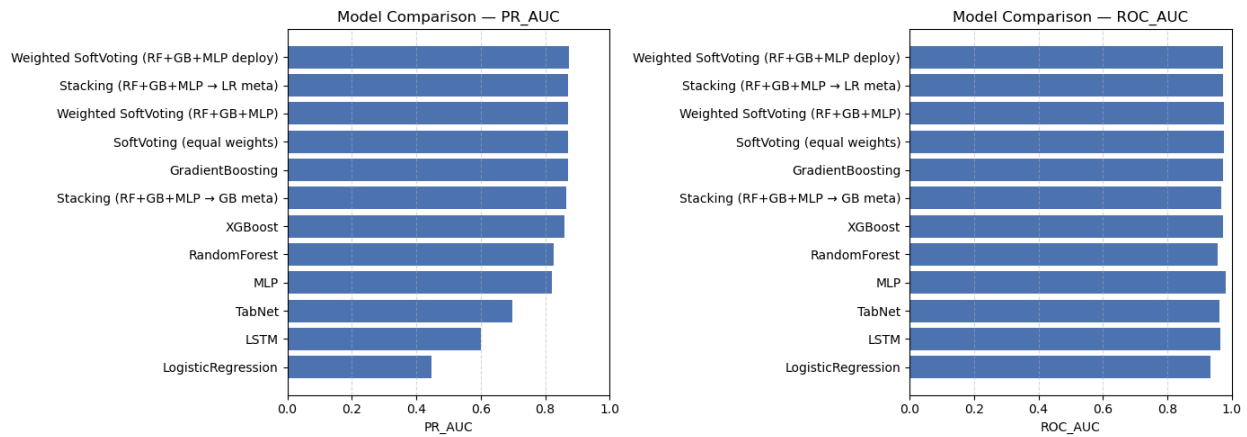


Figure 3: Ensemble model comparison showing PR AUC and Recall side by side

SMOTE Imbalance Experiment

We investigated whether SMOTE could improve failure detection compared to `class_weight = "balanced"`. SMOTE improved recall (from 0.53 to 0.71) but halved precision and actually lowered PR AUC. The project adopted `class_weight = "balanced"` as the primary imbalance handling strategy, combined with threshold tuning rather than data level resampling.

SHAP Model Explainability

We applied SHAP analysis to explain both global feature importance and individual prediction reasoning.

Because the deployed model is a Soft Voting ensemble, Kernel SHAP (KernelExplainer) was applied using a background sample of 100 training observations, with explanations computed for 300 test observations. Explainability is particularly critical in predictive maintenance applications, as engineers must understand the reasons behind high predicted failure risk to make informed preventive actions. Model-agnostic approaches such as SHAP provide theoretically grounded,

consistent feature attributions that enhance trust and operational decision-making in safety- and cost-critical environments (Adadi & Berrada, 2018).

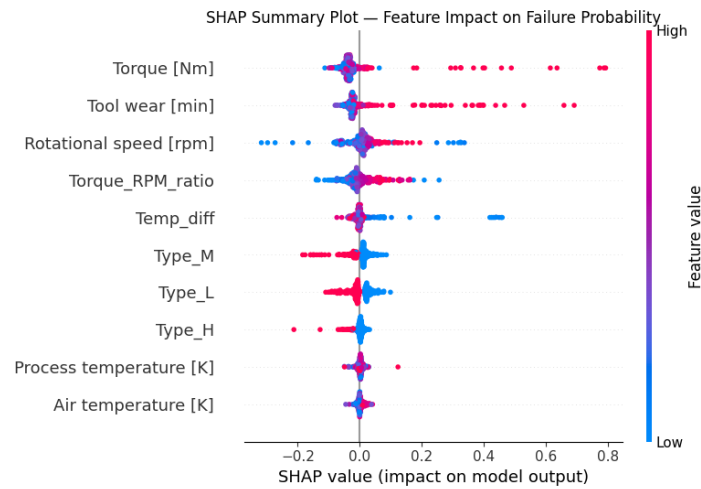


Figure 4: SHAP summary plot of key features affecting failure probability.

Torque and tool wear emerged as the two most influential features globally (Mean $|\text{SHAP}|$ of 0.0593 and 0.0542 respectively), consistent with the statistical testing results from Chapter III. The engineered Torque_RPM_ratio feature ranked fourth, validating its inclusion. SHAP dependence plots revealed that the relationship between torque and failure risk is not linear, with risk increasing sharply only beyond a certain torque threshold.

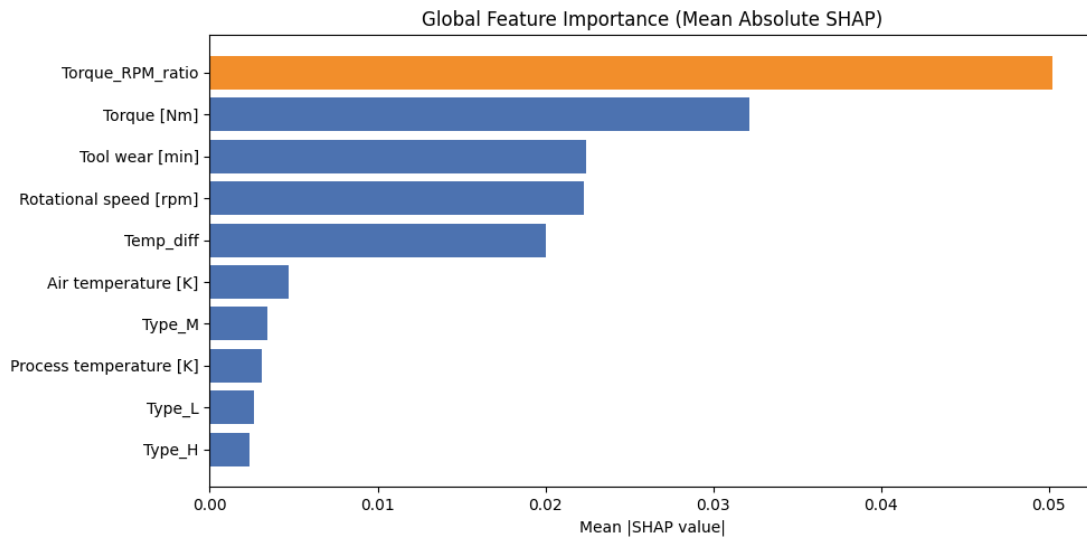


Figure 5: SHAP Summary Plot showing global feature importance and directionality

RUL/TTF Regression Estimation

Estimating Remaining Useful Life (RUL) allows maintenance teams to anticipate how long a machine can operate before failure, enabling more efficient scheduling of maintenance activities. While classification predicts whether a machine is likely to fail, it does not provide temporal information about failure onset (Lei et al., 2018). To address this gap, Notebook 09 trained a regression model to estimate RUL in minutes. A synthetic RUL target was constructed as ***RUL = wear_remaining/degradation_rate*** ***RUL = wear_remaining/degradation_rate***, where the degradation rate is a composite stress score combining torque (0.35), wear (0.25), RPM (0.20), and temperature (0.20). These weights were assigned based on statistical effect sizes obtained from hypothesis testing, with torque given the highest weight due to the largest Cohen's d. Among candidate models, the Gradient Boosting Regressor achieved the lowest mean absolute error (MAE) and was selected for deployment.

Sensitivity validation confirmed that the model responds correctly to sensor changes: increasing torque decreases predicted TTF, increasing tool wear decreases TTF, and increasing RPM increases TTF.

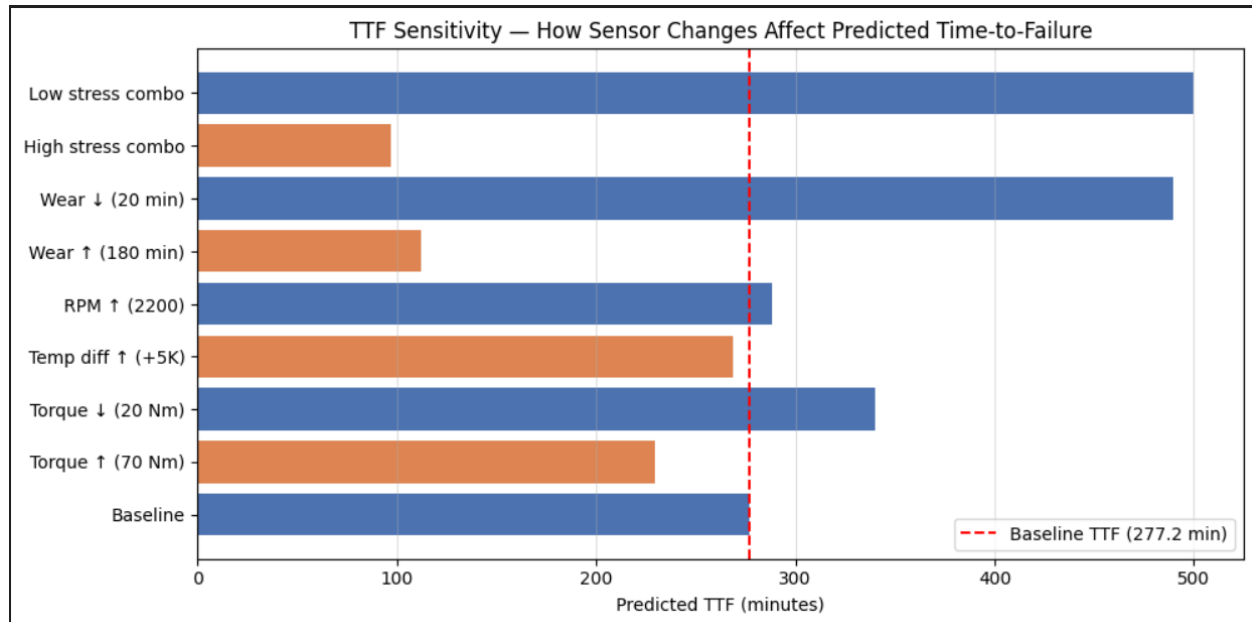


Figure 6: SHAP Summary Plot showing global feature importance and directionality

Survival Analysis

Survival analysis provides a probabilistic framework for estimating when a machine is likely to fail and which operational variables influence that timing (Kaplan and Meier, 1958). Notebook 10 used tool wear as a proxy for operational time. Kaplan Meier curves at the population level showed distinct survival profiles across machine types, and the log rank test confirmed that these differences are statistically significant ($p = 0.0058$).

Three separate Cox Proportional Hazards models were fitted by machine type rather than one overall model, because the failure dynamics differ across quality tiers (Cox, 1972). Air temperature

consistently showed the strongest effect: HR = 2.93 for Type H ($p = 0.0007$), HR = 2.43 for Type M ($p < 0.001$), and HR = 1.32 for Type L ($p < 0.001$). Torque showed a consistent HR of 1.19 to 1.24 across all types. Process temperature showed a protective effect in all three models, likely because once air temperature is controlled, the residual variation captures different operating regimes rather than thermal risk.

Two deep learning survival models, DeepSurv (Katzman et al., 2018) and Cox NNet, were compared against Cox PH using the concordance index. Cox NNet scored highest (0.9492), followed by DeepSurv (0.9408) and Cox PH (0.9080). Despite this, Cox PH was retained for the dashboard because it provides per covariate hazard ratios that operators can interpret directly, something neither deep learning model can offer. The 0.041 gap does not justify losing that transparency.

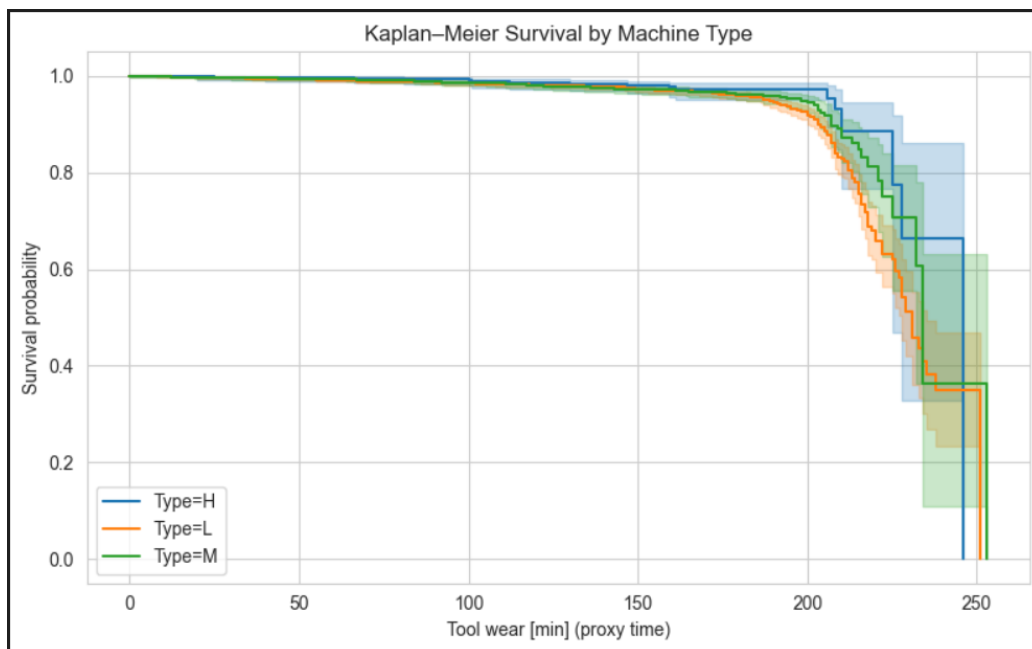


Figure 7: Kaplan Meier Survival Curves Stratified by Machine Type

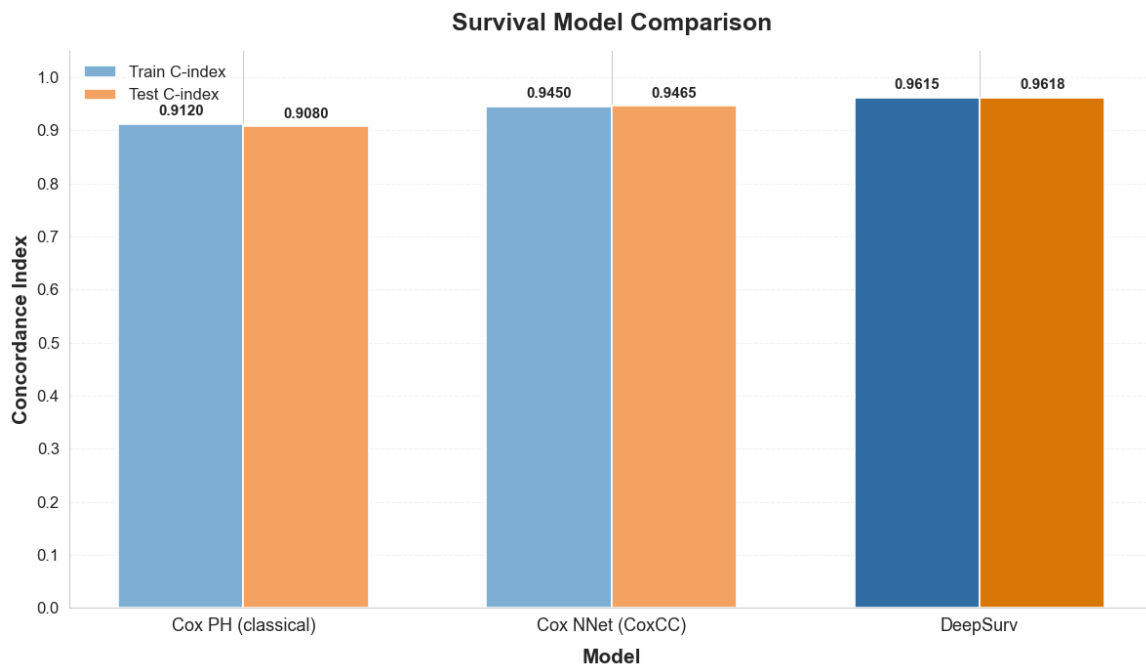


Figure 8: Survival Model Comparison

The analytical techniques described in this chapter provide multiple perspectives on machine failure prediction and risk assessment. The next chapter presents visual representations of these results through exploration plots, model evaluation charts, and the interactive predictive maintenance dashboard.

Chapter V: Data Visualization

This chapter presents visual representations of the analytical results obtained from the predictive maintenance models. Visualizations are used to illustrate patterns in the data, evaluate model performance, and communicate key insights in an intuitive manner. The figures also demonstrate how the developed dashboard enables interactive exploration of machine failure risk.

Streamlit Dashboard Overview

All analytical outputs were consolidated into a three-page interactive Streamlit dashboard, designed to serve as a real time monitoring and decision support tool for maintenance operators. The dashboard integrates the classification model, RUL regression, SHAP explainability, and survival analysis into a unified interface. The application is built with a modular architecture where the `src/` directory contains fourteen Python modules, each responsible for a specific function. This separation of concerns ensures that each component can be tested, maintained, and updated independently.

Page 1: Simulation Dashboard

The simulation page is the operational core of the system. It provides a real-time aging simulation where all selected machines degrade simultaneously in the background, not just the one currently being viewed. This was an important design decision because in a real factory, every machine is aging at the same time regardless of whether anyone is looking at it.

When an operator selects a Product ID from the sidebar, the dashboard loads that machine's current sensor readings and runs them through the prediction pipeline. A risk gauge displays the failure

probability as a percentage, color coded with green (0 to 35%), yellow (35 to 70%), and red (70 to 100%) zones. Four KPI cards show the Product ID, risk probability, risk level label, and estimated Time to Failure in minutes. A SHAP drivers table shows exactly which sensor variables are pushing the risk up or down for the currently selected machine.

A risk probability trend chart tracks how the selected machine's predicted risk evolves over simulation ticks. The y axis scales dynamically so that low risk machines still show meaningful visual movement rather than being compressed at the bottom of a 0 to 100% scale.

Automatic Maintenance at 90% Risk

One of the most important operational features we added to the dashboard is the automatic maintenance trigger. When any machine predicts failure, probability reaches or exceeds 90%, the system automatically places that machine into a maintenance state. We chose 90% rather than a lower threshold because we wanted to simulate a realistic scenario where maintenance is initiated only when the risk becomes genuinely critical, not merely elevated.

While a machine is under maintenance, its sensor simulation is paused. It does not continue aging or degrading during the maintenance period. The duration of maintenance varies by machine type: Type H machines require 20 simulation ticks, Type M machines require 15 ticks, and Type L machines require 12 ticks. We set these durations differently because higher quality machines (Type H) tend to have more complex components that take longer to service.

When maintenance is complete, the machine's sensors are reset to healthy baseline values. Tool wear is set back to zero (representing a fresh tool installation). Torque and rotational speed are reset to the fleet wide median values computed from the original dataset. Air and process

temperatures return to their original baseline levels. This simulates a real-world maintenance event where worn parts are replaced, and the machine is returned to normal operating condition.

Table 9: Maintenance Duration by Machine Type

Machine Type	Maintenance Ticks	Trigger Threshold
Type H (High)	20 ticks	90%
Type M (Medium)	15 ticks	90%
Type L (Low)	12 ticks	90%

High Risk Alerts and Maintenance Tables

Below the risk trend chart, two fleet level monitoring tables provide operators with a complete picture of the plant's health status. The High-Risk Alerts table automatically surfaces all machines that have crossed the 70% failure probability threshold but have not yet reached the 90% maintenance trigger. This table shows the Product ID, Type, risk probability, TTF estimate, and key sensor readings for each flagged machine, sorted from highest risk to lowest. This gives maintenance planners a prioritized inspection list.

The Machines Under Maintenance table shows all machines that have been automatically pulled into maintenance because they crossed the 90% threshold. This table includes the Product ID, Type, and the number of maintenance ticks remaining before the machine is returned to service. Together, these two tables transform the dashboard from a single machine monitoring tool into a fleet level operations center.



Figure 9: Fleet monitoring tables showing high risk machines and maintenance status

CSV Snapshot Save and Resume

We added a snapshot persistence feature to address a practical limitation of Streamlit applications: when the browser refreshes or the user pauses the simulation, all simulated machine states would normally be lost. This would be frustrating for an operator who has been running a simulation for several minutes and wants to pause, review results, and continue later.

When the operator clicks Pause, the dashboard automatically saves the current state of all 10,000 machines to a CSV file (`data/latest_snapshot.csv`). This snapshot includes the current sensor readings for every machine in the fleet, not just the ones being actively simulated. Simulated machines get their live sensor values, and a model predicted failure label. Non simulated machines retain their values from the previous snapshot or from the original cleaned dataset.

When the operator clicks Start again, the dashboard checks whether a snapshot file exists and loads it if available. This means the simulation picks up exactly where it left off, with all machines at their last known state. The sidebar displays a data source indicator showing whether the current data comes from the snapshot, the original dataset, or a mix of both.

The Reset button clears the snapshot file, removes all simulated states from memory, and reverts the dashboard to the original cleaned dataset. This gives the operator a clean slate to start a fresh simulation from the original data.

Machine Selection

The dashboard includes a multiselect widget that allows the operator to choose which machines to simulate. Rather than running all 10,000 machines simultaneously (which would be computationally expensive), the operator can select a subset of machines of interest. This makes the simulation more practical for focused analysis while still maintaining the ability to monitor the full fleet through the snapshot system.

Page 2: What If Analysis

The What If page allows operators to manually edit any machine's sensor values and immediately see how the predictions change. The entire dataset is presented in an editable table with failure mode columns hidden to prevent leakage. After making edits, the operator clicks Done to commit changes, which triggers a full re prediction and updates all KPIs and the risk gauge. This page is particularly valuable for scenario planning.

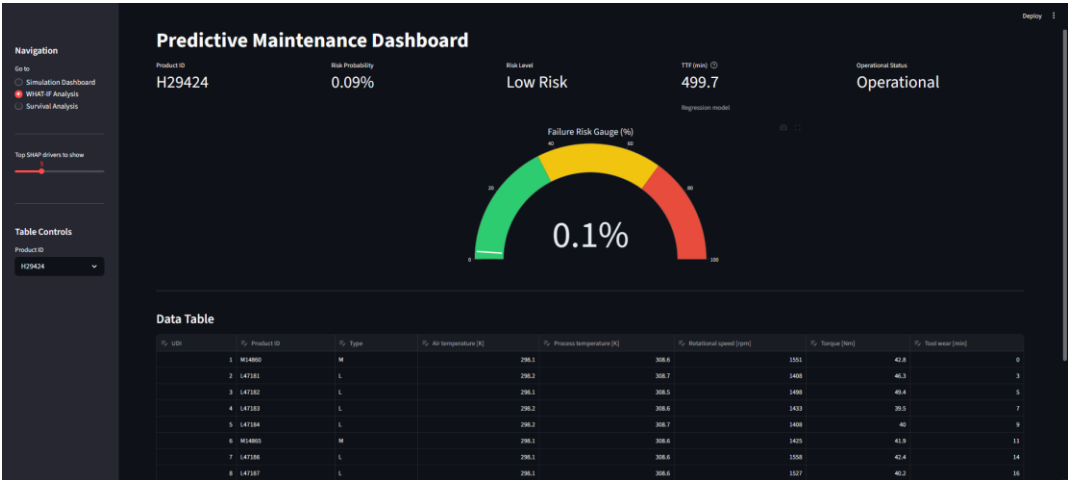


Figure 10: What If Analysis with editable sensor values

Page 3: Survival Analysis

The survival analysis page presents Kaplan Meier and Cox PH results side by side. The left panel shows population level KM survival curves with configurable cohort filtering (by machine type, wear bin, or both). The right panel shows the Cox predicted individual survival curve for the selected machine based on its actual sensor values. Two collapsible sections provide additional detail: a hazard ratios table and a Cox coefficients plot.

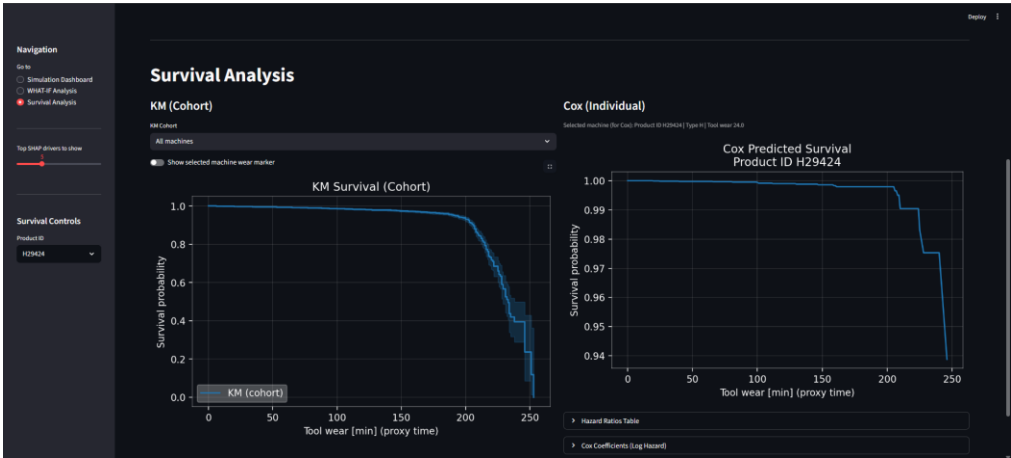


Figure 11: Survival Analysis with KM and Cox panels

Chapter VI: Findings and Discussion

Findings

The central research question asked which machine learning approach provides the best precision recall tradeoff for predicting machine failure under class imbalance. The answer, supported by systematic comparison across eight models, is that ensemble methods, specifically Soft Voting with calibrated base learners, deliver the strongest overall performance.

The Weighted Soft Voting with calibrated RF ensemble achieved a PR AUC of 0.873, with the tuned threshold, the model catches over 95% of real failures at the cost of approximately 10% false alarm rate among safe machines.

H1 (Temperature and Load Interaction): Supported. Logistic Regression with Interaction Term showed p value 0.0116 (<0.05)

H2 (Sensor Association): Strongly supported. One way ANOVA and Welch ANOVA test showed all 5 sensors statistically significant ($p < 0.05$)

H3 (Model Performance): Supported. Friedman test confirms Ensemble models achieve a mean PR-AUC of 0.8650 which is 93.8% improvement over Logistic Regression (0.4463) and outperforming deep learning models (mean PR-AUC 0.7583)

Discussion

The most striking finding is the enormous gap between Logistic Regression and tree-based models. Logistic Regression achieved high recall (0.868) but abysmal precision (0.175), flagging nearly

every machine as potentially at risk. The engineered Torque_RPM_ratio feature was designed to help capture nonlinear interactions, and its high SHAP ranking (4th overall) validates this intuition.

The SMOTE experiment yielded a counterintuitive result: synthetic oversampling lowered PR AUC compared to the baseline, suggesting that the original failure distribution is informative, and SMOTE generated samples may introduce noise.

The automatic maintenance feature at 90% risk adds an important operational dimension that goes beyond pure prediction. It simulates a complete maintenance lifecycle: detection, intervention, repair, and return to service. This demonstrates how the predictive model integrates into actual maintenance workflows rather than existing as an isolated scoring tool.

The snapshot persistence feature addresses a practical limitation that would otherwise make the dashboard unsuitable for real use. Without it, any browser refresh would erase minutes or hours of simulated degradation.

One important limitation is that the AI4I 2020 dataset lacks a true temporal dimension. Tool wear serves as a proxy for operational time, and the RUL regression target was synthetically constructed. The degradation rate weights (0.35 for torque, 0.25 for wear, 0.20 each for RPM and temperature) were informed by statistical effect sizes but not optimized through cross validation.

Chapter VII: Conclusion and Recommendations

Conclusion

This project demonstrated that a comprehensive, data driven predictive maintenance system can be built from publicly available industrial sensor data using open-source tools. Starting from 10,000 machine observations with a 3.39% failure rate, our team developed a full analytics pipeline that spans data cleaning, statistical validation, multi model comparison, explainability analysis, time to failure estimation, survival modeling, and interactive dashboard deployment.

The Weighted Soft Voting with calibrated RF ensemble serves as the core prediction engine, catching over 95% of real failures while maintaining manageable false alarm rates. The three-page Streamlit dashboard brings all analytical layers together in a single interface with practical operational features: automatic maintenance triggering at 90% risk, fleet level high risk alerts, machines under maintenance tracking, CSV snapshot persistence for pause and resume capability, and configurable machine selection for focused simulation.

This work aligns with Industry 4.0 principles by demonstrating how intelligent, connected, and data driven systems can transform industrial maintenance from a reactive cost center into a proactive operational advantage.

Recommendations

First, threshold selection should be guided by the specific cost structure of the deployment environment. The 95% recall target used here assumes missed failures are significantly more expensive than false alarms.

Second, model explainability should be treated as a core requirement. The SHAP integration provides the transparency that maintenance engineers need to trust and act on the system's predictions.

Third, survival analysis should be included alongside binary predictions because knowing that a machine's predicted survival drops below 50% at 180 minutes of wear is more actionable than a probability score alone.

Fourth, real time sensor streaming would significantly enhance the system's value by connecting to actual sensor feeds rather than simulated data.

Fifth, the degradation rate weights in the RUL model could be improved through systematic optimization using cross validation or Bayesian hyperparameter tuning rather than manual assignment based on effect sizes.

Chapter VIII: Project Management

Agile Methodology and Taiga.io

This project followed the Agile software development methodology with Taiga.io as the project management platform and GitHub for version control. The work was organized into user stories, each representing a specific deliverable or analytical task, with story points to estimate effort. The Scrum framework was applied with defined sprint goals and regular check ins.

Sprint Overview

The project was divided into four sprints spanning seven weeks. The first three sprints were each two weeks long, while the fourth sprint was a one-week buffer.

Table 10: Agile Sprint Timeline

Sprint	Focus	Dates	Duration	Stories
Sprint 1	Data Foundation and EDA	31 Jan to 13 Feb 2026	2 weeks	32 / 32
Sprint 2	Advanced Modeling and Evaluation	14 Feb to 27 Feb 2026	2 weeks	32 / 32
Sprint 3	Dashboard and System Integration	28 Feb to 13 Mar 2026	2 weeks	32 / 32
Sprint 4	Buffer and Safety Margin	14 Mar to 20 Mar 2026	1 week	22/22

Sprint 1 focused on understanding the dataset, performing exploratory data analysis, completing data cleaning, and establishing the hypothesis testing framework. Sprint 2 covers baseline model training, ensemble model comparison, SMOTE experimentation, threshold optimization, and SHAP explainability. Sprint 3 delivered the Streamlit dashboard with all three pages, the RUL regression model, survival analysis integration, maintenance logic, snapshot persistence, and

system testing. Sprint 4 was reserved for final bug fixes, report writing, and presentation preparation.

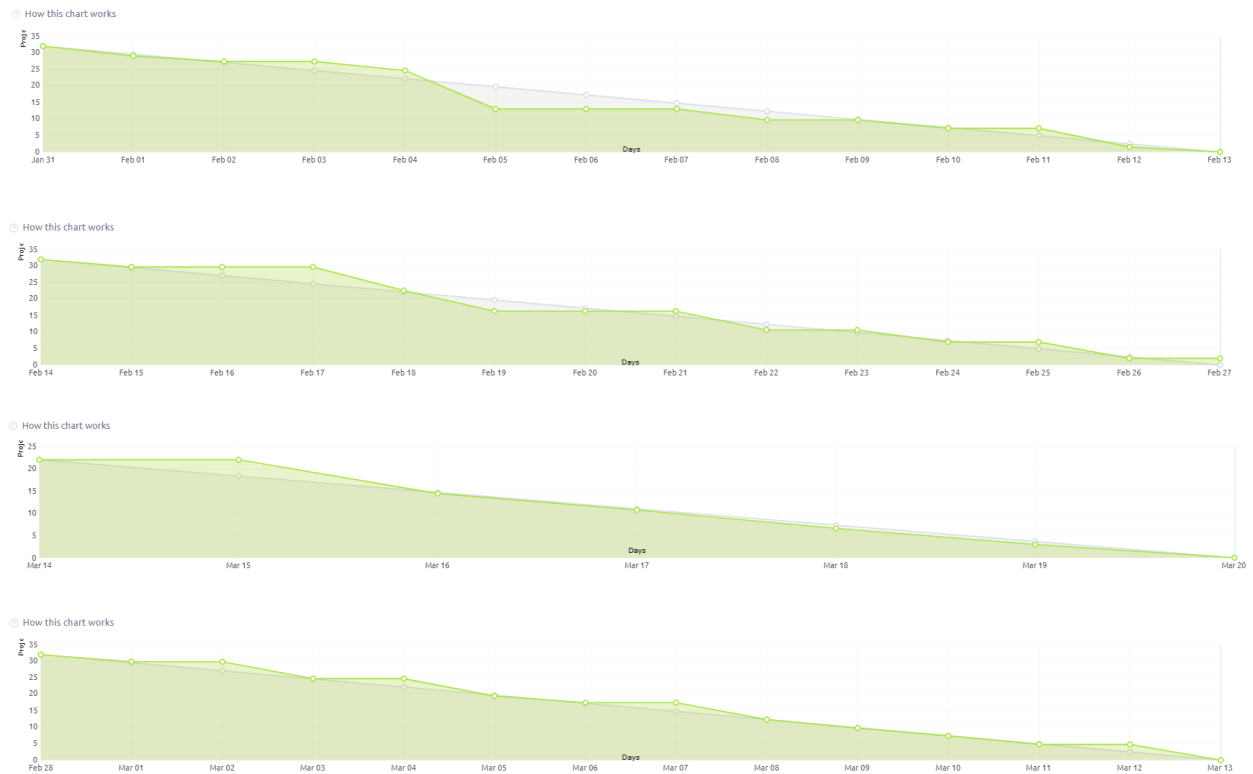


Figure 12: Taiga Sprint Taskboard Overview

Version Control with GitHub

The team used GitHub for version control with a branch-based workflow. All source code, notebooks, model artifacts, and documentation were maintained in a shared repository (Capstone_Project_Damo-699). User stories in Taiga mapped directly to branches and commits in GitHub, creating clear traceability from planning to implementation.

References

- Adadi, A., & Berrada, M. (2018). Peeking inside the black-box: A survey on explainable artificial intelligence (XAI). *IEEE Access*, 6, 52138–52160.
<https://doi.org/10.1109/ACCESS.2018.2870052>
- Carvalho, T. P., Soares, F. A. A. M. N., Vita, R., Francisco, R. P., Basto, J. P., & Alcala, S. G. S. (2019). A systematic literature review of machine learning methods applied to predictive maintenance. *Computers & Industrial Engineering*, 137, Article 106024.
<https://doi.org/10.1016/j.cie.2019.106024>
- Cotton Gins. (2022, February 16). *Cost analysis: Predictive maintenance vs. reactive repairs*.
<https://cottongins.org/blog/cost-analysis-predictive-maintenance-vs-reactive-repairs>
- Cox, D. R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2), 187–220. <https://doi.org/10.1111/j.2517-6161.1972.tb00899.x>
- Davidson-Pilon, C. (2019). Lifelines: Survival analysis in Python. *Journal of Open Source Software*, 4(40), 1317. <https://doi.org/10.21105/joss.01317>
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- Géron, A. (2019). *Hands-on machine learning with Scikit-learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
<https://www.deeplearningbook.org>

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>

Kaggle. (2020). *Predictive maintenance dataset (AI4I 2020)* [Data set]. <https://www.kaggle.com/datasets/stephanmatzka/predictive-maintenance-dataset-ai4i-2020>

Kaplan, E. L., & Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 457–481. <https://doi.org/10.1080/01621459.1958.10501452>

Lei, Y., Li, N., Guo, L., Li, N., Yan, T., & Lin, J. (2018). Machinery health prognostics: A review. *Mechanical Systems and Signal Processing*, 104, 799–834. <https://doi.org/10.1016/j.ymssp.2017.11.016>

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). <https://arxiv.org/abs/1705.07874>

Mobley, R. K. (2002). *An introduction to predictive maintenance* (2nd ed.). Butterworth-Heinemann.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Zhu, Z., Dong, M., Huang, B., & Chen, Q. (2019). *A survey of predictive maintenance: Systems, methods, and approaches*. arXiv. <https://arxiv.org/abs/1912.07383>

Zonta, T., da Costa, C. A., da Rosa Righi, R., de Lima, M. J., da Trindade, E. S., & Li, G. P. (2020). Predictive maintenance in the industry 4.0: A systematic literature review. *Computers & Industrial Engineering*, 150, Article 106889. <https://doi.org/10.1016/j.cie.2020.106889>

Appendix A: Project Repository Structure

The project codebase follows a modular architecture with clear separation between data, source code, notebooks, dashboard, and model artifacts:

Capstone_Project_Damo-699/

data/

raw/ai4i2020.csv

cleaned/ai4i2020_cleaned.csv

latest_snapshot.csv (auto generated on pause)

src/

__init__.py

data_loader.py (Raw and cleaned data loading)

features.py (Engineered features: Temp_diff, Torque_RPM_ratio)

preprocessing.py (sklearn ColumnTransformer builder)

labeling.py (Risk tier assignment: Low/Medium/High)

train.py (Full training pipeline: Weighted Soft Voting with calibrated RF + threshold + RUL)

inference.py (Prediction + RUL estimation)

shap_explain.py (SHAP driver extraction)

simulator.py (Realistic sensor state simulation)

survival_analysis.py (KM, Cox PH, log rank tests)

ttf_trend.py (Trend based TTF estimation)

dashboard_utils.py (Dashboard helpers: gauge, KPIs, maintenance, snapshot)

generate_visuals.py (Batch chart generation for visuals/ folder)

dashboard/

streamlit_app.py (Three page Streamlit application)

notebooks/ (01 through 10, covering EDA to survival analysis)

models/

best_model.pkl (Deployed Soft Voting ensemble)

rul_regressor.joblib (RUL regression model)

threshold.json (Tuned decision threshold)

eval.json (Evaluation metrics)

visuals/ (All charts saved as PNG files)